Tabla de contenidos

1. Serialización de objetos
    a. `pickle`
        i. Personalizar la serialización en `pickle`
        ii. Personalizar la deserialización en `pickle`

Serialización de objetos

Continuando con lo que vimos la semana pasada sobre serialización. Python además incluye otra forma de serializar los datos a bytes.

`pickle`

El módulo `pickle` de Python permite guardar y cargar casi cualquier objeto de Python. Este módulo ofrece los siguientes métodos:

- `dumps`: serializa un objeto, es decir, lo **guarda** como una secuencia de bytes.
- `loads`: deserializa un objeto serializado, es decir, **carga** un objeto a su estado original.

Una vez que un objeto es serializado, este es persistente y está listo para volver a ser usado en el futuro por el mismo u otro programa.

```
import pickle


tupla = ("a", 1, 3, "hola")
tupla_serializada = pickle.dumps(tupla)

print(f"Resultado serialización: {tupla_serializada}")
print(f"Tipo de versión serializada: {type(tupla_serializada)}")

tupla_deserializada = pickle.loads(tupla_serializada)
print(f"Resultado deserialización: {tupla_deserializada}")

print()
print(f"Tupla original: {tupla}")
print(f"Tupla deserializada: {tupla_deserializada}")
print(f"¿Las tuplas son iguales? {tupla == tupla_deserializada}")
print(f"¿Las tuplas son el mismo objeto? {tupla is
tupla_deserializada}")
```

```
Resultado serialización: b'\x80\x04\x95\x13\x00\x00\x00\x00\x00\x00\
x00(\x8c\x01a\x94K\x01K\x03\x8c\x04hola\x94t\x94.'
Tipo de versión serializada: <class 'bytes'>
Resultado deserialización: ('a', 1, 3, 'hola')
```

```
Tupla original: ('a', 1, 3, 'hola')
Tupla deserializada: ('a', 1, 3, 'hola')
¿Las tuplas son iguales? True
¿Las tuplas son el mismo objeto? False
```

Además, `pickle` nos ofrece los métodos `dump` y `load` (casi el mismo nombre que los anteriores, pero sin la *s*). Estos métodos también serializan y deserializan, pero **a través de archivos**:

- `dump`: guarda un archivo con el objeto serializado.
- `load`: deserializa un objeto almacenado en un archivo.

Nótese, acá como estamos trabajando con bytes en vez de *strings*, los archivos debemos abrirlos en modo binario (`b`) en vez de texto (que es el modo por defecto).

```
from os import path

lista = [1, 2, 3, 7, 8, 3]

with open(path.join("data", "mi_lista.bin"), "wb") as file:
    pickle.dump(lista, file)

with open(path.join("data", "mi_lista.bin"), "rb") as file:
    lista_cargada = pickle.load(file)

print(f"Lista original: {lista}")
print(f"Lista cargada : {lista_cargada}")
print(f"¿Las listas son iguales? {lista == lista_cargada}")
print(f"¿Las listas son el mismo objeto? {lista is lista_cargada}")
```

```
Lista original: [1, 2, 3, 7, 8, 3]
Lista cargada : [1, 2, 3, 7, 8, 3]
¿Las listas son iguales? True
¿Las listas son el mismo objeto? False
```

Importante

`pickle` es un módulo no seguro. Esto significa que **nunca** debes cargar un archivo *pickle* cuando no conoces su procedencia, ya que éste podría ejecutar código malicioso en tu computador. No entraremos en detalles sobre cómo inyectar código a través del módulo `pickle`, pero si te interesa puedes revisar este enlace donde se demuestra su uso malicioso.

Personalizar la serialización en `pickle`

Cuando `pickle` trata de serializar un objeto, lo primero que hará es verificar que el objeto que se quiere serializar sea de una clase que tenga implementado el método `__getstate__`. Este método debe retornar un diccionario con los atributos que se quieren serializar. Si `__getstate__` no estuviese implementado, entonces `pickle` guardará el atributo `__dict__` del objeto.

El atributo `__dict__` es un diccionario que guarda todos los atributos y métodos de un objeto. En otras palabras, `objeto.atributo` es equivalente a `objeto.__dict__["atributo"]` y `objeto.atributo = 42` es equivalente a `objeto.__dict__["atributo"] = 42`.

El implementar el método `__getstate__` nos permite personalizar la serialización del objeto. Usando este método podemos crear un diccionario que contenga solo la información que deseamos guardar.

```
class Persona:

    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre
        self.edad = edad
        self.mensaje = "No pasa nada"

    def __getstate__(self) -> dict:
        """
        Retorna el estado actual del objeto, para que sea serializado
por pickle

        Aquí creamos una copia del diccionario actual, para modificar
la copia
        y no el objeto original
        """
        # Usamos una copia del diccionario original para alterar solo
la copia
        nueva = self.__dict__.copy()
        # Modificamos un atributo en el objeto serializado.
        # Sin embargo, el objeto original no ha cambiado.
        nueva.update({"mensaje": "¡Me están serializando!"})
        # Lo que retornemos es lo que será serializado por pickle
        return nueva

original = Persona("Juan", 30)
print(f"Mensaje original: {original.mensaje}")
serializado = pickle.dumps(original)
deserializado = pickle.loads(serializado)

# El objeto original sigue igual
print(f"Mensaje original: {original.mensaje}")
print(f"Mensaje deserializado: {deserializado.mensaje}")

Mensaje original: No pasa nada
Mensaje original: No pasa nada
Mensaje deserializado: ¡Me están serializando!
```

Personalizar la deserialización en `pickle`

De forma análoga, podemos personalizar la **deserialización**. Para esto debemos implementar el método `__setstate__`, que se ejecutará cada vez que llamemos a `load` o `loads`. El método

`__setstate__` recibe como argumento el diccionario que representa el estado del objeto que fue serializado. Luego debe asignarlo al diccionario del objeto `self.__dict__ = diccionario_con_estado`. Esto no impide que se realicen otras acciones que modifiquen `diccionario_con_estado` antes o después de la asignación.

Si el método `__setstate__` no estuviese implementado, entonces se asignará al `__dict__` del objeto el estado deserializado sin realizar otras acciones adicionales.

```
class Persona:

    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre
        self.edad = edad
        self.mensaje = "No pasa nada"

    def __getstate__(self) -> dict:
        nueva = self.__dict__.copy()
        print(f"[__getstate__] Serializando a {nueva['nombre']}")
        nueva.update({"mensaje": "¡Me están serializando!"})
        # Lo que retornemos es lo que será serializado por pickle
        return nueva

    def __setstate__(self, state) -> None:
        print("[__setstate__] Objeto recién deserializado,
actualizando su estado")
        # Al desarializar modificamos el estado
        state.update({"nombre": f"{state['nombre']} deserializado"})
        self.__dict__ = state

original = Persona("Juan", 30)
print(f"Nombre original: {original.nombre}")
# Al usar pickle.dumps() se ejecuta el método __getstate__
serializado = pickle.dumps(original)
```

```
Nombre original: Juan
[__getstate__] Serializando a Juan
```

```
print("¡Ejecutar loads → deserializar!")
# Al usar pickle.loads() se ejecuta el método __setstate__
deserializado = pickle.loads(serializado)
print(f"Nombre deserializado: {deserializado.nombre}, y su mensaje:
{deserializado.mensaje}")
```

```
¡Ejecutar loads → deserializar!
[__setstate__] Objeto recién deserializado, actualizando su estado
Nombre deserializado: Juan deserializado, y su mensaje: ¡Me están
serializando!
```

Una aplicación de los métodos `__getstate__` y `__setstate__` es cuando necesitamos serializar un objeto que contiene un atributo que depende de las condiciones actuales del programa.

Por ejemplo, imaginemos que un objeto que guarda información sobre los usuarios conectados actualmente al programa, como la cantidad de usuarios y la información correspondiente a la conexión con cada uno. Cuando guardamos el objeto, deberíamos eliminar estas conexiones, ya que al cargarlo en otra instancia del programa no deberíamos poder comunicarnos con los usuarios de la instancia anterior. Para lograr esto usamos el método `__getstate__`.

Similarmente, cuando se cargue el mismo objeto desde el archivo serializado, será necesario volver a crear las conexiones con las condiciones del programa nuevo. Para realizar esto tendremos que implementar `__setstate__`.